Deadline-Aware Online Optimization for Mobile Edge Computing via Age-Layered Queues

Xin Gao*, Hongwei Shi†*, Chen Su*, Zhi Zheng* and Tao Huang‡*
*Future Network Center, Purple Mountain Laboratories, Nanjing 211111, China
†School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China
‡State Key Laboratory of Networking and Switching Technology,
Beijing Univ. of Posts and Telecom. (BUPT), Beijing 100876, China
Emails: {gaoxin, shihongwei, suchen, zhengzhi, huangt}@pmlabs.com.cn

***Abstract*—Abstract: Mobile edge computing (MEC) faces challenges in meeting the stringent, deterministic delay requirements of mission-critical 6G services due to stochastic dynamics and coupled resources. This paper investigates online resource allocation for MEC networks under hard end-to-end delay constraints. We develop an age-layered queue model with age-priority service to capture and process workloads based on urgency. We then formulate a long-term stochastic optimization problem to minimize energy consumption and deadline-violation penalties. To solve it, we propose the age-layered deterministic-delay optimization (ADDO) algorithm. Based on Lyapunov drift-plus-penalty theory, ADDO jointly optimizes local computing, offloading, and edge CPU allocation without requiring prior traffic or channel statistics. Theoretical analysis shows an $O(1/V)$ cost-optimality gap. Simulations demonstrate that ADDO achieves superior deadline reliability over benchmarks and can eliminate violations with proper parameter tuning.**

***Index Terms*—edge computing, deterministic delay, online optimization, Lyapunov optimization, resource allocation.**

I. Introduction

With the rapid development of artificial intelligence, the Internet of Things (IoT), and next-generation mobile communications, future 6G networks are expected to support mission-critical services such as autonomous driving, industrial automation, and immersive interaction [1]. These applications require high data rates, ubiquitous connectivity, and stringent end-to-end delay guarantees. Mobile edge computing (MEC), which moves computation and storage resources closer to users [2], is a key paradigm for latency-sensitive services. However, in stochastic MEC systems, task arrivals, wireless channels, local computing, and edge CPU resources are time-varying and tightly coupled, making hard deadline provisioning much more challenging than conventional average-delay optimization.

Deterministic latency has been widely studied in communication networks. Time-Sensitive Networking (TSN) and Deterministic Networking (DetNet) provide bounded-latency forwarding through scheduling, synchronization, and resource reservation [3], while ultra-reliable low-latency communication (URLLC) studies delay–reliability tradeoffs based on finite blocklength coding, effective capacity, and stochastic network calculus [4], [5]. These works provide foundations for predictable transmission latency, but mainly focus on the communication layer and do not explicitly address end-to-end deterministic latency in MEC systems, where transmission, offloading, queueing, and edge execution are jointly involved.

Latency-aware resource allocation has also been extensively investigated in MEC, including joint task offloading and resource allocation for delay optimization [6], risk-aware formulations based on chance constraints or CVaR [7], [8], queueing-theoretic delay analysis [9], and robust optimization under uncertainty [10]. To handle time-varying arrivals, channel states, and computing demands, stochastic optimization, Markov decision processes, reinforcement learning, and Lyapunov-based methods have been applied to online MEC control [11]–[15]. Among them, Lyapunov optimization is attractive because it enables online decisions without prior statistical knowledge. Nevertheless, most existing MEC frameworks optimize time-average cost, average delay, probabilistic delay violation, or virtual-queue-based surrogate metrics, and thus do not directly expose the urgency of individual workloads for hard end-to-end deadline enforcement.

To bridge this gap, this paper studies online resource allocation for MEC networks under deterministic delay constraints. We develop an age-layered queue model that partitions both local and edge backlogs by workload age, so that workloads approaching their maximum admissible age can be explicitly identified. An age-priority service discipline is further adopted, under which older workloads are served first in local computing, task offloading, and edge execution. This design makes deadline urgency directly observable in both local and edge queues, allowing the controller to distinguish accumulated workloads from those close to deadline violation.

Based on the proposed model, we formulate a long-term stochastic optimization problem to minimize the average system cost, including local computing energy, transmission energy, edge computing energy, and deadline-violation penalties, subject to queue stability and resource constraints. We then propose an age-layered deterministic-delay optimization (ADDO) algorithm based on Lyapunov drift-plus-penalty optimization. ADDO jointly determines local CPU frequencies, uplink transmit powers, and edge CPU allocation in each slot without requiring prior traffic-arrival or channel-state statistics. The local computing and transmission decisions have closed-form solutions, while the coupled edge CPU allocation is efficiently solved by a bisection-based dual method.

The main contributions are threefold. First, we develop an age-layered queueing framework that explicitly tracks workload urgency in both local and edge queues for MEC systems with hard end-to-end delay constraints. Second, we formulate a stochastic online optimization problem that jointly captures communication cost, computation cost, and deadline-violation penalties under time-varying arrivals and wireless channels. Third, we propose ADDO and establish its theoretical guarantees, including an $O(1/V)$ cost-optimality gap and an $O(V)$ average-backlog tradeoff, where $V > 0$ is the Lyapunov control parameter. Numerical results show that ADDO achieves the lowest deadline-violation ratio among benchmark policies and can eliminate deadline violations in certain scenarios.

The remainder of this paper is organized as follows. Section II presents the system model. Section III formulates the optimization problem. Section IV develops the proposed algorithm and theoretical analysis. Section V presents numerical results, and Section VI concludes the paper.

II. System Model

We consider a time-slotted MEC network with a set of users $\mathcal{K} = \{1, \dots, K\}$ and one edge server[1]. Each slot $t \in \{0, 1, \dots\}$ has duration δ. Users can process workloads locally or offload them to the co-located edge server.

A. Workload and Resource Model

For each user $i \in \mathcal{K}$, let $A_i(t)$ denote the workload arriving at the beginning of slot t, measured in bits.[2] The stochastic and time-varying arrival process is bounded by $0 \le A_i(t) \le A_{i,\max}, \forall t$. Newly arrived workloads are first buffered and then scheduled for local or edge processing. Each workload of user i has a hard end-to-end deadline $D_i \in \mathbb{Z}_+$, requiring completion within D_i slots after arrival.[3]

Let c_i be the CPU cycles per bit for user i. Local and edge CPU frequencies, $f_i^{\mathrm{L}}(t)$ and $f_i^{\mathrm{E}}(t)$, yield computing capacities (maximum bits per slot):

$$\mu_i^{\mathrm{L}}(t) = \delta f_i^{\mathrm{L}}(t)/c_i, \quad \mu_i^{\mathrm{E}}(t) = \delta f_i^{\mathrm{E}}(t)/c_i, \tag{1}$$

The frequencies are subject to:

$$0 \le f_i^{\mathrm{L}}(t) \le f_{i,\max}^{\mathrm{L}}, \quad \forall i, t, \tag{2}$$

$$\sum_{i \in \mathcal{K}} f_i^{\mathrm{E}}(t) \le f_{\max}^{\mathrm{E}}, \quad f_i^{\mathrm{E}}(t) \ge 0, \quad \forall i, t, \tag{3}$$

where $f_{i,\max}^{\mathrm{L}}$ and $f_{\max}^{\mathrm{E}}$ denote the maximum local and total edge CPU capacities, respectively.

Each user may offload part of its local workload through the wireless uplink. Let $p_i(t)$ denote the transmit power of user i in slot t, constrained by

$$0 \le p_i(t) \le p_{i,\max}, \quad \forall i, t. \tag{4}$$

[1]The proposed framework can be extended to multi-server MEC systems by augmenting the offloading action space and server-side queue states.

[2]The workload can also be measured in packets or tasks after normalization.

[3]To make the model meaningful, we assume $D_i \ge 2$ for each user $i \in \mathcal{K}$.

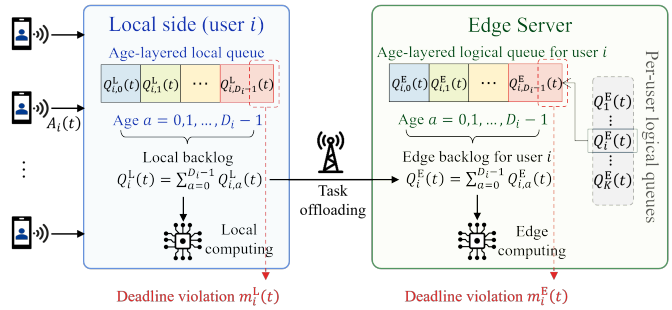


Fig. 1: Age-layered queue model.

The offloading capacity is

$$\mu_i^{\mathrm{T}}(t) = \delta B_i \log_2 \left(1 + p_i(t) H_i(t)/(N_0 B_i)\right), \tag{5}$$

where $H_i(t)$ is the channel power gain, B_i is the bandwidth, and N_0 is the noise power spectral density. We model $H_i(t) = \beta_0 d_i(t)^{-\alpha} \xi_i(t)$ to capture both large-scale path loss and small-scale fading, where $d_i(t)$ is the user-edge distance, β_0 is the reference gain at $d_0 = 1$ m, and α is the path-loss exponent. The fading term is $\xi_i(t) = |g_i(t)|^2$ with $g_i(t) \sim \mathcal{CN}(0, 1)$. We assume a frequency-flat block fading channel, where $H_i(t)$ remains constant per slot and varies independently across slots.

B. Age-Layered Queue Model

Each user $i \in \mathcal{K}$ maintains a local queue $Q_i^{\mathrm{L}}(t)$, while the edge server manages K logical queues $Q_i^{\mathrm{E}}(t)$ to facilitate user-specific scheduling under a shared computation budget. New workloads enter the local queue for local processing or offloading. Offloaded tasks arrive at the edge server in slot $t+1$, accounting for a one-slot transmission latency; downlink feedback delay is neglected due to small output sizes.

To enforce delay deadline D_i, we adopt an age-layered representation with age index $a \in \{0, \dots, D_i - 1\}$. A workload in layer a has resided in the system for a slots. Any workload in layer $D_i - 1$ not completed within the current slot violates the deadline and is either removed or urgently processed.

Let $Q_{i,a}^{\mathrm{L}}(t)$ and $Q_{i,a}^{\mathrm{E}}(t)$ denote the age-a local and edge workloads, respectively. The total local and edge backlogs are aggregated as $Q_i^{\mathrm{L}}(t) = \sum_{a=0}^{D_i-1} Q_{i,a}^{\mathrm{L}}(t)$ and $Q_i^{\mathrm{E}}(t) = \sum_{a=0}^{D_i-1} Q_{i,a}^{\mathrm{E}}(t)$, respectively. Fig. 1 illustrates the proposed age-layered queueing architecture.

C. Age-priority Service Discipline

Let $x_{i,a}(t)$, $u_{i,a}(t)$, and $y_{i,a}(t)$ denote the amount of age-a workload processed locally, offloaded, and processed at the edge, respectively. These satisfy capacity constraints: $\sum_{a=0}^{D_i-1} x_{i,a}(t) = X_i(t) \le \mu_i^{\mathrm{L}}(t)$, $\sum_{a=0}^{D_i-1} u_{i,a}(t) = U_i(t) \le \mu_i^{\mathrm{T}}(t)$, and $\sum_{a=0}^{D_i-1} y_{i,a}(t) = Y_i(t) \le \mu_i^{\mathrm{E}}(t)$.

We adopt an *age-priority service discipline* (analogous to Earliest-Deadline-First), where older workloads are served first across all computing and offloading resources. Under this rule, local, transmission, and edge capacities are allocated to layers in descending order of age a. Given aggregate capacities and

queue states, the age-wise variables $\{x_{i,a}(t), u_{i,a}(t), y_{i,a}(t)\}$ are uniquely determined.

1) Local computing: For each user i, local computing is performed in descending age order. For $a = D_i - 1$,

$$x_{i,D_i-1}(t) = \min\{Q^{\mathrm{L}}_{i,D_i-1}(t), \mu^{\mathrm{L}}_i(t)\},\ \forall i, t, \tag{6}$$

and for $a = D_i - 2, D_i - 3, \ldots, 0$,

$$x_{i,a}(t) = \min\{Q^{\mathrm{L}}_{i,a}(t), \mu^{\mathrm{L}}_i(t) - X^{D_i-1}_{i,a+1}(t)\},\ \forall i, t. \tag{7}$$

where $X^{D_i-1}_{i,a+1}(t) = \sum_{\alpha=a+1}^{D_i-1} x_{i,\alpha}(t)$.

2) Offloading: After local computing, the residual local backlog is $\hat{Q}^{\mathrm{L}}_{i,a}(t) = Q^{\mathrm{L}}_{i,a}(t) - x_{i,a}(t)$. To avoid deadline violation during transmission, age-$(D_i - 1)$ workloads are not offloaded, i.e., $u_{i,D_i-1}(t) = 0$ for all i, t. The remaining eligible workloads are offloaded in descending age order. For $a = D_i - 2$,

$$u_{i,D_i-2}(t) = \min\{\hat{Q}^{\mathrm{L}}_{i,D_i-2}(t), \mu^{\mathrm{T}}_i(t)\},\ \forall i, t, \tag{8}$$

and for $a = D_i - 3, D_i - 4, \ldots, 0$,

$$u_{i,a}(t) = \min\{\hat{Q}^{\mathrm{L}}_{i,a}(t), \mu^{\mathrm{T}}_i(t) - U^{D_i-2}_{i,a+1}(t)\},\ \forall i, t. \tag{9}$$

where $U^{D_i-2}_{i,a+1}(t) = \sum_{\alpha=a+1}^{D_i-2} u_{i,\alpha}(t)$.

3) Edge computing: At the edge side, each logical edge queue is also served in descending age order. For $a = D_i - 1$,

$$y_{i,D_i-1}(t) = \min\{Q^{\mathrm{E}}_{i,D_i-1}(t), \mu^{\mathrm{E}}_i(t)\},\ \forall i, t, \tag{10}$$

and for $a = D_i - 2, D_i - 3, \ldots, 0$,

$$y_{i,a}(t) = \min\{Q^{\mathrm{E}}_{i,a}(t), \mu^{\mathrm{E}}_i(t) - Y^{D_i-1}_{i,a+1}(t)\},\ \forall i, t. \tag{11}$$

where $Y^{D_i-1}_{i,a+1}(t) = \sum_{\alpha=a+1}^{D_i-1} y_{i,\alpha}(t)$.

D. Deadline-Violation Modeling

For user i, the local and edge deadline-violation amounts in slot t are defined as $m^{\mathrm{L}}_i(t) = Q^{\mathrm{L}}_{i,D_i-1}(t) - x_{i,D_i-1}(t)$ and $m^{\mathrm{E}}_i(t) = Q^{\mathrm{E}}_{i,D_i-1}(t) - y_{i,D_i-1}(t)$, respectively. These quantities measure the workload bits that miss the deterministic delay bound in slot t. Such overdue workloads are either removed or forcibly completed through urgent computation. Each violated bit incurs penalties $\phi^{\mathrm{L}}_i > 0$ and $\phi^{\mathrm{E}}_i > 0$ at the local and edge sides, respectively, yielding the total violation cost $\phi^{\mathrm{L}}_i m^{\mathrm{L}}_i(t) + \phi^{\mathrm{E}}_i m^{\mathrm{E}}_i(t)$.

E. Queue Evolution

1) Local queue dynamics: New arrivals enter the age-0 local queue at the beginning of each slot, i.e., $Q^{\mathrm{L}}_{i,0}(t) = A_i(t)$ for all i, t. During slot t, age-a workloads are locally processed by $x_{i,a}(t)$ or offloaded by $u_{i,a}(t)$, while the remaining workloads age by one slot. Thus, for $a = 0, 1, \ldots, D_i - 2$,

$$Q^{\mathrm{L}}_{i,a+1}(t+1) = Q^{\mathrm{L}}_{i,a}(t) - x_{i,a}(t) - u_{i,a}(t). \tag{12}$$

Aggregating all age layers, the total local backlog evolves as

$$Q^{\mathrm{L}}_i(t+1) = Q^{\mathrm{L}}_i(t) + A_i(t+1) - m^{\mathrm{L}}_i(t) - X_i(t) - U_i(t). \tag{13}$$

2) Edge queue dynamics: Since no workload is generated at the edge, $Q^{\mathrm{E}}_{i,0}(t) = 0$ for all i, t. The workload $u_{i,a}(t)$ offloaded from local age layer a in slot t arrives at the edge server at the beginning of slot $t+1$ with age $a+1$, while $y_{i,a}(t)$ is processed at the edge. Therefore, for $a = 0, 1, \ldots, D_i - 2$,

$$Q^{\mathrm{E}}_{i,a+1}(t+1) = Q^{\mathrm{E}}_{i,a}(t) - y_{i,a}(t) + u_{i,a}(t). \tag{14}$$

Aggregating all age layers, the total edge backlog evolves as

$$Q^{\mathrm{E}}_i(t+1) = Q^{\mathrm{E}}_i(t) + U_i(t) - m^{\mathrm{E}}_i(t) - Y_i(t). \tag{15}$$

F. Feasible Control Policy

In each slot t, the controller observes the network state $\mathbf{\Theta}(t) \triangleq \{Q^{\mathrm{L}}_{i,a}(t), Q^{\mathrm{E}}_{i,a}(t), A_i(t), H_i(t),\ \forall i, a\}$ and makes online decisions on local CPU frequencies, transmit powers, and edge CPU allocation, denoted by $\chi(t) \triangleq \{f^{\mathrm{L}}_i(t), p_i(t), f^{\mathrm{E}}_i(t),\ \forall i\}$. The age-wise service variables $\{x_{i,a}(t), u_{i,a}(t), y_{i,a}(t),\ \forall i, a\}$ are then implicitly determined by the age-priority service discipline.

The online optimization objective is to adapt $\chi(t)$ to stochastic arrivals and time-varying channels while satisfying deterministic delay requirements. Depending on the design target, the system may minimize long-term communication-computation energy consumption, edge resource usage, or network-wide utility loss, while preserving queue feasibility and deadline compliance.

III. Problem Formulation

We aim to design an online control policy that determines $\chi(t)$ in each slot to minimize the long-term system cost while satisfying deterministic delay requirements.

A. Per-Slot Cost Function

In each slot t, the system cost consists of energy cost and deadline-violation cost:

$$C(t) = \sum_{i\in\mathcal{K}} \Big(C_i(t) + \phi^{\mathrm{L}}_i r^{\mathrm{L}}_i(t) + \phi^{\mathrm{E}}_i r^{\mathrm{E}}_i(t) \Big), \tag{16}$$

where $C_i(t) = C^{\mathrm{L}}_i(t) + C^{\mathrm{T}}_i(t) + C^{\mathrm{E}}_i(t)$ is the sum of local computing, uplink transmission, and edge computing costs for processing or delivering user i's workload. Specifically,

$$C^{\mathrm{L}}_i(t) = \delta\kappa (f^{\mathrm{L}}_i(t))^3, C^{\mathrm{E}}_i(t) = \delta\kappa (f^{\mathrm{E}}_i(t))^3, C^{\mathrm{T}}_i(t) = \delta p_i(t), \tag{17}$$

where $\kappa > 0$ is the effective switched-capacitance coefficient.

B. Long-Term Optimization Problem

The long-term time-average system cost and total queue backlog are defined as

$$\bar{C} = \limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[C(t)\right], \tag{18}$$

$$\bar{Q} = \limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i\in\mathcal{K}} \left(\mathbb{E}\left[Q^{\mathrm{L}}_i(t)\right] + \mathbb{E}\left[Q^{\mathrm{E}}_i(t)\right] \right). \tag{19}$$

Let $\{\chi(t)\}_{t=0}^{\infty}$ be feasible control actions adapted to the observed network state $\mathbf{\Theta}(t)$. The long-term stochastic optimization problem is formulated as

$$\textbf{(P1)} \quad \min_{\{\chi(t)\}_{t=0}^{\infty}} \quad \bar{C}$$
$$\text{s.t.} \quad \bar{Q} < \infty, \ (2), \ (3), \ (4),$$
$$\text{Age-priority service discipline (Sec. II-C).}$$

where the expectation is taken over stochastic arrivals and wireless channel states. Note that **(P1)** involves stochastic dynamics and coupled decisions.

IV. Algorithm Design

Using Lyapunov optimization [16], we decompose (P1) into per-slot subproblems. Local computing and transmission decisions are user-decoupled, whereas edge CPU allocation remains coupled by the shared resource constraint. Detailed algorithm derivations are available in our technical report [].

A. Local CPU optimization

The local CPU allocation for user i is obtained by solving

$$\min_{f_i^{\mathrm{L}}(t)} \quad VC_i^{\mathrm{L}}(t) + \varphi_i^{\mathrm{L}}(t) r_i^{\mathrm{L}}(t) - Q_i^{\mathrm{L}}(t)\mu_i^{\mathrm{L}}(t)$$
$$\text{s.t.} \quad (1)(2).$$

where $\varphi_i^{\mathrm{L}}(t) = V\phi_i^{\mathrm{L}} - Q_i^{\mathrm{L}}(t)$. The optimal solution is

$$\tilde{f}_i^{\mathrm{L}}(t) = \arg \min_{f \in \{f_{i,1}^{\mathrm{L}}(t),\, f_{i,2}^{\mathrm{L}}(t)\}} J_i^{\mathrm{L}}(f), \tag{20}$$

where $J_i^{\mathrm{L}}(f) = \varphi_i(t)[f_{i,\mathrm{th}}^{\mathrm{L}}(t) - f]^+/c_i + V\kappa f^3 - Q_i^{\mathrm{L}}(t) f/c_i$, $f_{i,1}^{\mathrm{L}}(t) = [\sqrt{\phi_i^{\mathrm{L}}/(3\kappa c_i)}]_0^{\min\{f_i^{\max},\, f_{i,\mathrm{th}}^{\mathrm{L}}(t)\}}$, and $f_{i,2}^{\mathrm{L}}(t) = [\sqrt{Q_i^{\mathrm{L}}(t)/(3V\kappa c_i)}]_{f_{i,\mathrm{th}}^{\mathrm{L}}(t)}^{f_i^{\max}}$. Here, $f_{i,\mathrm{th}}^{\mathrm{L}}(t) = c_i Q_{i,D_i-1}^{\mathrm{L}}(t)/\delta$ is the threshold value, $[\cdot]^+ = \max\{\cdot, 0\}$, and $[\cdot]_a^b \triangleq \min\{\max\{\cdot, a\}, b\}$.

B. Transmission power optimization

For each user i, define $\Delta Q_i(t) = Q_i^{\mathrm{E}}(t) - Q_i^{\mathrm{L}}(t)$, then the transmission power is obtained by solving

$$\min_{p_i(t)} \quad VC_i^{\mathrm{T}}(t) + \Delta Q_i(t)\mu_i^{\mathrm{T}}(t)$$
$$\text{s.t.} \quad (4)(5).$$

The optimal solution is obtained in closed form as

$$\tilde{p}_i(t) = [\Delta Q_i(t) B_i/(V \ln 2) - N_0 B_i/H_i(t)]_0^{p_{i,\max}}. \tag{21}$$

C. Edge CPU allocation

The edge CPU allocation is obtained by solving

$$\min_{\{f_i^{\mathrm{E}}(t)\}} \sum_{i\in\mathcal{K}} VC_i^{\mathrm{E}}(t) + \varphi_i^{\mathrm{E}}(t) r_i^{\mathrm{E}}(t) - Q_i^{\mathrm{E}}(t)\mu_i^{\mathrm{E}}(t)$$
$$\text{s.t.} \ (1)(3).$$

where $\varphi_i^{\mathrm{E}}(t) = V\phi_i^{\mathrm{E}} - Q_i^{\mathrm{E}}(t)$. Introducing the dual variable $\lambda \geq 0$ for the coupling constraint $\sum_{i\in\mathcal{K}} f_i^{\mathrm{E}}(t) \leq f_{\max}^{\mathrm{E}}$, the Lagrangian is:

$$\mathcal{L}\big(\{f_i^{\mathrm{E}}(t)\}, \lambda\big) = \sum_{i\in\mathcal{K}} \Big(J_i^{\mathrm{E}}\big(f_i^{\mathrm{E}}(t)\big) + \lambda f_i^{\mathrm{E}}(t) \Big) - \lambda f_{\max}^{\mathrm{E}}, \tag{22}$$

Algorithm 1 Bisection Search for Optimal Dual

Initialize $\lambda_{\min} = 0$, $\lambda_{\max} = \max_i \frac{\delta}{c_i} \max\{V\phi_i^{\mathrm{E}}, Q_i^{\mathrm{E}}(t)\}$, and tolerance $\epsilon > 0$.
repeat
  Set $\lambda = (\lambda_{\min} + \lambda_{\max})/2$.
  Compute $\tilde{f}_i^{\mathrm{E}}(t;\lambda)$ according to (23) for each $i \in \mathcal{K}$.
  Set $\lambda_{\min} = \lambda$ if $\sum_{i\in\mathcal{K}} \tilde{f}_i^{\mathrm{E}}(t;\lambda) > f_{\max}^{\mathrm{E}}$, and set $\lambda_{\max} = \lambda$ otherwise.
until $|\lambda_{\max} - \lambda_{\min}| < \epsilon$
Set $\lambda^* = \lambda$ and output $\tilde{f}_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t;\lambda^*)$, $\forall i \in \mathcal{K}$.

Algorithm 2 ADDO Algorithm

for each slot $t = 0, 1, 2, \ldots$ **do**
  Compute λ^* via Algorithm 1.
  for each user $i \in \mathcal{K}$ **do**
    Compute $\tilde{f}_i^{\mathrm{L}}(t)$, $\tilde{p}_i(t)$, $\tilde{f}_i^{\mathrm{E}}(t;\lambda^*)$ according to (20), (21), and (23), respectively.
    Set $f_i^{\mathrm{L}}(t) = \tilde{f}_i^{\mathrm{L}}(t)$, $p_i(t) = \tilde{p}_i(t)$, $f_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t;\lambda^*)$.
    Serve following the age-priority service discipline.
  end for
  Update queues.
end for

where $J_i^{\mathrm{E}}(f) = \varphi_i^{\mathrm{E}}(t)[f_{i,\mathrm{th}}^{\mathrm{E}} - f]^+/c_i + V\kappa f^3 - Q_i^{\mathrm{E}}(t) f/c_i$. Here, $f_{i,\mathrm{th}}^{\mathrm{E}} = c_i Q_{i,D_i-1}^{\mathrm{E}}(t)/\delta$ is the threshold value.

For a given λ, the problem is decomposed into a set of independent one-dimensional subproblems. The optimal solution of the ith subproblem is given by

$$\tilde{f}_i^{\mathrm{E}}(t;\lambda) = \arg \min_{f \in \{f_{i,1}^{\mathrm{E}}(\lambda),\, f_{i,2}^{\mathrm{E}}(\lambda)\}} \big(J_i^{\mathrm{E}}(f) + \lambda f \big), \tag{23}$$

where $f_{i,1}^{\mathrm{E}}(\lambda) = [\sqrt{[V\delta\phi_i^{\mathrm{E}}/c_i - \lambda]^+/(3V\delta\kappa)}]_0^{f_{i,\mathrm{th}}^{\mathrm{E}}}$ and $f_{i,2}^{\mathrm{E}}(\lambda) = [\sqrt{[\delta Q_i^{\mathrm{E}}(t)/c_i - \lambda]^+/(3V\delta\kappa)}]_{f_{i,\mathrm{th}}^{\mathrm{E}}}^{+\infty}$.

The optimal solution is $\tilde{f}_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t;\lambda^*)$, where $\lambda^* \geq 0$ satisfies $\sum_{i\in\mathcal{K}} \tilde{f}_i^{\mathrm{E}}(t;\lambda^*) \leq f_{\max}^{\mathrm{E}}$ with complementary slackness. Given the monotonicity of the sum in λ, λ^* is efficiently determined via bisection search in Algorithm 1.

The complete ADDO algorithm is summarized in Algorithm 2. Its per-slot complexity is $O(K)$ for local CPU and power allocation, and $O(K\log(1/\epsilon))$ for edge CPU allocation via bisection.

D. Performance Guarantee

The following theorems characterize ADDO's performance.

Theorem 1 (Cost and Backlog Bound): Assume arrival $\{A_i(t), \ \forall i, t\}$ lies in the interior of the capacity region of the system. Let C^* denote the minimum time-average expected cost of problem **(P1)** over all feasible policies. Then, under ADDO with control parameter $V > 0$, the following holds:

$$\bar{C} \leq C^* + B/V, \tag{24}$$
$$\bar{Q} \leq \min\{(B + VC_{\max})/\varepsilon, Q_{\max}\}, \tag{25}$$

where $B, \varepsilon > 0$, $Q_{\max} = \sum_{i\in\mathcal{K}} D_i A_{i,\max}$, and $C_{\max} \triangleq \sum_{i\in\mathcal{K}}(\phi_i^{\mathrm{L}}+\phi_i^{\mathrm{E}})A_{i,\max}+\delta\sum_{i\in\mathcal{K}} p_{i,\max}+\delta\kappa\sum_{i\in\mathcal{K}}(f_{i,\max}^{\mathrm{L}})^3+\delta\kappa(f_{\max}^{\mathrm{E}})^3$ is a finite per-slot cost upper bound.

Theorem 2 (Deadline-Violation Bound): Define the time-average expected deadline-violation amount as

$$\bar{M} \triangleq \limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{i\in\mathcal{K}} \mathbb{E}\big[m_i^L(t)+m_i^E(t)\big]. \tag{26}$$

Let $\phi_{\min} \triangleq \min_{i\in\mathcal{K}}\{\phi_i^L, \phi_i^E\} > 0$, then under ADDO,

$$\bar{M} \le \bar{C}/\phi_{\min} \le (C^* + B/V)/\phi_{\min}. \tag{27}$$

We provide all details and proofs in our technical report [].

V. Numerical Results

A. Simulation Setup

We consider a stochastic MEC system with one edge server and 20 users. Time is slotted with duration $\delta = 10$ ms. For each user i, the workload arrivals are i.i.d. with $A_i(t) \sim \mathrm{Unif}(0, 3\times 10^4)$ bits, and each bit requires $c_i \sim \mathrm{Unif}(500, 1000)$ CPU cycles. All tasks share a common deterministic delay bound $D = 4$ slots unless otherwise specified.

For the wireless uplink, we set the reference channel gain $\beta_0 = 10^{-4}$, path-loss exponent $\alpha = 4$, user-edge distance 150 m, noise power spectral density $N_0 = -174$ dBm/Hz, bandwidth $B_i = 1$ MHz, and maximum transmit power $p_{i,\max} = 100$ mW. Each user has a local CPU with $f_{i,\max}^L = 1$ GHz, while the edge server provides a shared CPU capacity of $f_{\max}^E = 10$ GHz. The DVFS coefficient is $\kappa = 10^{-28}$, and the deadline-violation penalties are $\phi_i^L = \phi_i^E = 0.01$.

We compare ADDO with four baselines: the *delay-aware energy-efficient (DAEE) scheme* [11], which uses virtual queues and perturbed Lyapunov optimization to guarantee task deadlines; *LocalOnly*, where all workloads are processed locally; *EdgeOnly*, where all workloads are offloaded to the edge server; and *Random*, which uniformly allocates communication and computation resources over the feasible action space. The performance is evaluated by average energy cost, deadline-violation ratio, and average delay per bit.

B. Experimental Results

1) Comparison with baseline policies: Table I compares ADDO with four baselines, where $V = 10^{11}$ for both ADDO and DAEE. ADDO achieves the lowest deadline-violation ratio and average delay among all schemes. Compared with DAEE, it reduces the energy cost from 0.0341 to 0.0296 J/slot, i.e., by about 13.20%, while eliminating deadline violations and slightly reducing delay.

Although LocalOnly, EdgeOnly, and Random may consume less energy than ADDO, they suffer significant reliability degradation. For example, LocalOnly has the lowest energy cost of 0.0200 J/slot, 32.43% lower than ADDO, but incurs a deadline-violation ratio of 0.1544 and an average delay of 3.0029 slots/bit. In contrast, ADDO eliminates deadline violations and reduces the delay by 46.27%, indicating that purely local execution or full offloading cannot effectively handle stochastic arrivals and communication–computation coupling under hard deadline constraints.

TABLE I: Performance Comparison of Different Policies

Policy	Energy Cost (J/slot)	Deadline Violation Ratio	Average Delay (slots/bit)
ADDO	0.0296	**0.0000**	**1.6135**
DAEE	0.0341	0.0142	1.6276
LocalOnly	**0.0200**	0.1544	3.0029
EdgeOnly	0.0258	0.5642	3.9196
Random	0.0282	0.2373	2.8050

2) Comparison of ADDO and DAEE under different V: Fig. 2 compares ADDO and DAEE as V varies from 10^8 to 2×10^{11}. ADDO consistently achieves better deadline reliability than DAEE: its deadline-violation ratio remains nearly zero for $V \le 1.5\times 10^{11}$, and even at $V = 2\times 10^{11}$, it reduces the violation ratio from 0.0457 to 0.0176, i.e., by 61.49%.

As shown in Fig. 2b, ADDO also yields lower delay in the moderate-V regime. For example, at $V = 2\times 10^{10}$, it reduces the average delay from 1.5685 to 1.3930 slots, corresponding to an 11.20% reduction. However, excessively large V increases ADDO's delay more rapidly, suggesting that a moderate V better balances delay and deadline reliability.

Fig. 2c shows that the energy costs of both algorithms decrease with V and gradually saturate. In the small-V regime, ADDO is much more energy-efficient; at $V = 10^8$, it reduces the energy cost from 0.9378 to 0.2309 J/slot, i.e., by 75.39%. Overall, ADDO provides a more favorable reliability–delay–energy tradeoff than DAEE.

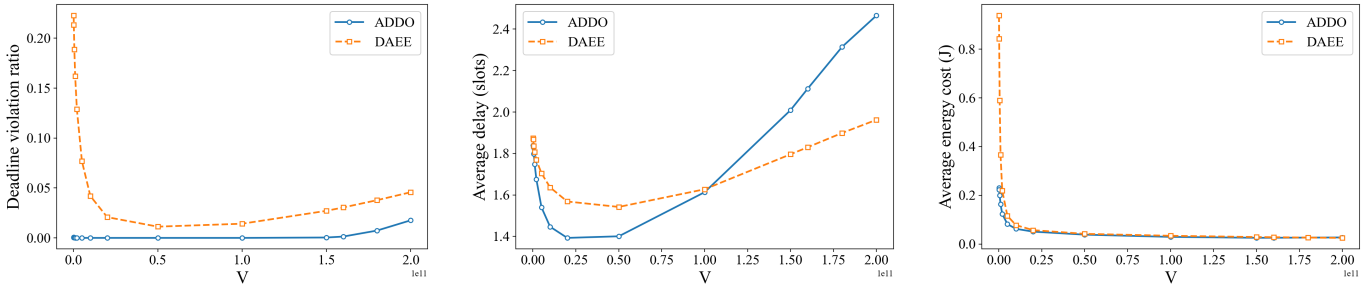
3) Performance versus the deterministic deadline D: Fig. 3 shows the performance of different policies as D varies from 2 to 10 slots, with $V = 10^{11}$ for ADDO and DAEE. As shown in Fig. 3a, ADDO achieves the best deadline reliability over the entire range of D: its violation ratio is about 0.0136 at $D = 2$ and becomes zero for $D \ge 3$, whereas DAEE has a violation ratio of about 0.1082 at $D = 2$ and only drops to 0.0001 when $D = 8$. LocalOnly, EdgeOnly, and Random still suffer high violation ratios even under relaxed deadlines.

Fig. 3b shows that ADDO maintains low delay as D increases. At $D = 10$, ADDO achieves an average delay of about 1.6256 slots, compared with 1.7201, 7.2597, 9.7731, and 5.5849 slots for DAEE, LocalOnly, EdgeOnly, and Random, respectively. This demonstrates that ADDO effectively suppresses queue buildup while satisfying deadline constraints.

As shown in Fig. 3c, the energy cost of ADDO decreases from about 0.0357 J/slot at $D = 2$ to about 0.0294 J/slot for $D \ge 5$, yielding a reduction of approximately 17.65%. ADDO also consumes less energy than DAEE over the entire range of D; for example, at $D = 10$, it reduces the energy cost from 0.0340 to 0.0294 J/slot, i.e., by about 13.53%. Overall, ADDO achieves a better delay–energy–reliability tradeoff than the benchmarks under different deterministic deadlines.

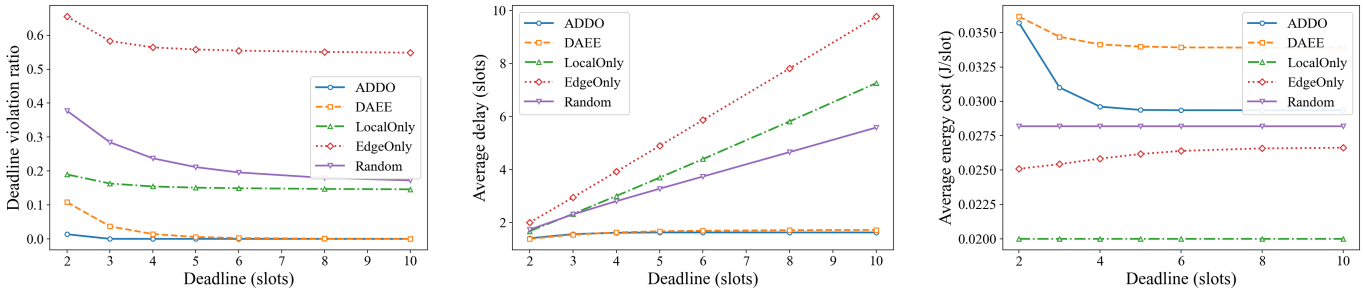
VI. Conclusion

This paper investigated online resource allocation in MEC networks under hard end-to-end delay constraints. By intro-



(a) Deadline violation ratio. (b) Average delay. (c) Average energy cost.

Fig. 2: Performance versus the Lyapunov parameter V under ADDO and DAEE.



(a) Deadline violation ratio. (b) Average delay. (c) Average energy cost.

Fig. 3: Performance versus the deterministic deadline D under different policies.

ducing an age-layered queue model with age-priority service, we formulated a stochastic optimization problem balancing energy consumption and deadline violations. An ADDO algorithm was developed to jointly optimize local computing, offloading, and edge CPU allocation without prior traffic or channel statistics. Analysis proved an $O(1/V)$ cost-optimality gap, and simulations showed that ADDO improves deadline reliability and the delay–energy tradeoff over benchmarks.

Acknowledgment

This work was supported in part by the National Science and Technology Major Project of China (Grant 2026ZD1307700) and the Science and Technology Major Project of Jiangsu Province (Grant BG2025003).

References

[1] C.-X. Wang, X. You, X. Gao, X. Zhu, Z. Li, C. Zhang, H. Wang, Y. Huang, Y. Chen, H. Haas *et al.*, "On the road to 6g: Visions, requirements, key technologies, and testbeds," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 905–974, 2023.

[2] M. Ergen, B. Saoud, I. Shayea, A. A. El-Saleh, O. Ergen, F. Inan, and M. F. Tuysuz, "Edge computing in future wireless networks: A comprehensive evaluation and vision for 6g and beyond," *ICT Express*, vol. 10, no. 5, pp. 1151–1173, 2024.

[3] H. Chahed and A. Kassler, "Tsn network scheduling—challenges and approaches," *Network*, vol. 3, no. 4, pp. 585–624, 2023.

[4] B. S. Khan, S. Jangsher, A. Ahmed, and A. Al-Dweik, "Urllc and embb in 5g industrial iot: A survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1134–1163, 2022.

[5] C. Xu, Y. Liu, C. Wang, and A. Nallanathan, "Analysis of grant-free access for urllc service: A stochastic network calculus perspective," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 9, pp. 14 949–14 954, 2025.

[6] J. Ren, G. Yu, Y. Cai, and Y. He, "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Transactions on Wireless Communications*, vol. 17, no. 8, pp. 5506–5519, 2018.

[7] C.-F. Liu, M. Bennis, M. Debbah, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Transactions on Communications*, vol. 67, no. 6, pp. 4132–4150, 2019.

[8] S. Yue, J. Ren, N. Qiao, Y. Zhang, H. Jiang, Y. Zhang, and Y. Yang, "Todg: Distributed task offloading with delay guarantees for edge computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 7, pp. 1650–1665, 2021.

[9] J. Cao, W. Feng, N. Ge, and J. Lu, "Delay characterization of mobile-edge computing for 6g time-sensitive services," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3758–3773, 2020.

[10] Q. Wang, X. Chen, and Q. Qi, "Design of delay-optimal robust edge computing in 6g wireless networks," in *IEEE WCSP*, 2022, pp. 226–231.

[11] H. Wu, J. Chen, T. N. Nguyen, and H. Tang, "Lyapunov-guided delay-aware energy efficient offloading in iiot-mec systems," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 2117–2128, 2022.

[12] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated mec systems: A distributed online approach," *IEEE Transactions on Mobile Computing*, vol. 23, no. 8, pp. 8129–8142, 2023.

[13] S. Xia, Z. Yao, Y. Li, Z. Xing, and S. Mao, "Distributed computing and networking coordination for task offloading under uncertainties," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5280–5294, 2024.

[14] L. Tang, M. Xu, C. Xu, and K. Ye, "Collaborative resource management and workloads scheduling in cloud-assisted mobile edge computing across timescales," in *IEEE ICWS*, 2024, pp. 1415–1425.

[15] S. Yao, M. Wang, J. Ren, T. Xia, W. Wang, K. Xu, M. Xu, and H. Zhang, "Multi-agent reinforcement learning for task offloading in crowd-edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9289–9302, 2025.

[16] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.